

P1238R1

SG16: Unicode Direction

Published Proposal, 2019-06-16

Authors:

[Tom Honermann](#)

[Corentin Jabot](#)

[JeanHeyd Meneide](#)

[Mark Zeren](#)

[Martinho Fernandes](#)

[Peter Bindels](#)

[Steve Downey](#)

[Zach Laine](#)

Audience:

DIRECTION, SG16

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

SG16 initial Unicode direction and guidance for C++20 and beyond.

Table of Contents

- 1 **Changes since [P1238R0]**
- 2 **Constraints: Accepting the things we cannot change**
 - 2.1 Constraint: The ordinary and wide execution encodings are implementation defined
 - 2.2 Constraint: The ordinary and wide execution encodings are run-time properties
 - 2.3 Constraint: There is no portable primary execution encoding
 - 2.4 Constraint: `wchar_t` is a portability deadend

2.5 Constraint: `char` aliases everything

2.6 Constraint: Implementors cannot afford to rewrite ICU

2.7 Constraint: File names do not have an associated character encoding

3 Guidelines: Keep your eyes on the road, your hands upon the wheel

3.1 Guideline: Avoid excessive inventiveness; look for existing practice

3.2 Guideline: Avoid gratuitous departure from C

4 Direction: Designing for where we want to be and how to get there

5 Directives: Do or do not, there is no try

5.1 Directive: Standardize new encoding aware text container and view types

5.2 Directive: Standardize generic interfaces for Unicode algorithms

5.3 Directive: Standardize useful features from other languages

5.4 Directive: Improve support for transcoding at program boundaries

5.5 Directive: Propose resolutions for existing issues and wording improvements opportunistically

6 Non-directives: Thanks, but No Thanks

6.1 Non-directive: User input

6.2 Non-directive: Fonts, graphical text rendering

6.3 Non-directive: Encoding of file names

7 Acknowledgements

References

Informative References

The SG16 Unicode study group was officially formed at the 2018 WG21 meeting in Jacksonville, Florida and had its inaugural meeting at the WG21 meeting in San Diego later that year. However, an active group of WG21 members has been meeting via video conference regularly since August of 2017, well before our formation as an official study group. Summaries of these meetings are available at [the SG16 meetings repository](#).

Our proposals so far have focused on relatively small or foundational features that had a realistic chance of being accepted for C++20. The following have been adopted for C++20:

- [P0482R6: char8_t: A type for UTF-8 characters and strings](#) [\[P0482R6\]](#)
- [P1025R1: Update The Reference To The Unicode Standard](#) [\[P1025R1\]](#)

- [P1041R4: Make char16_t/char32_t string literals be UTF-16/32](#) [\[P1041R4\]](#)

All other work that we are pursuing is targeting C++23 or later. Proposals so far include:

- [P1072R3: basic_string::resize_default_init](#) [\[P1072R3\]](#)
- [P1097R1: Named character escapes](#) [\[P1097R1\]](#)

This paper discusses a set of constraints, guidelines, directives, and non-directives intended to guide our continuing efforts to improve Unicode and text processing support in C++. Paper authors intending to propose Unicode or text processing related features are encouraged to consider the perspectives and guidelines discussed here in their designs, or to submit papers arguing against them.

§ 1. Changes since [\[P1238R0\]](#)

- Updated the introduction to reflect progress made for C++20.
- Updated constraint 1 with news of new compilers for z/OS that target newer language standards.
- Removed discussion of the encoding of char16_t and char32_t literals being implementation defined now that [P1041R4](#) [\[P1041R4\]](#) has been adopted for C++20.
- Added constraint 7, File names do not have an associated character encoding.
- Added non-directive 3, Encoding of file names.

§ 2. Constraints: Accepting the things we cannot change

C++ has a long history and, as unfortunate as it may be at times, the past remains stubbornly immutable. As we work to improve the future, we must remain cognizant of the many billions of lines of C++ code in use today and how we will enable past work to retain its value in the future. The following limitations reflect constraints on what we cannot affordably change, at least not in the short term.

§ 2.1. Constraint: The ordinary and wide execution encodings are implementation defined

[UTF-8 has conquered the web](#) [\[W3Techs\]](#), but no such convergence has yet occurred for the execution and wide execution character encodings. Popular and commercially significant platforms such as

Windows and z/OS continue to support a wide array of ASCII and EBCDIC based encodings for the execution character encoding as required for compatibility by their long time customers.

Might these platforms eventually move to UTF-8 or possibly cease to be relevant for new C++ standards?

Microsoft does not yet offer full support for UTF-8 as the execution encoding for its compiler. Support for a /utf-8 compiler option was added recently, but it does not affect the behavior of their standard library implementation, nor is UTF-8 selectable as the execution encoding at run-time via environment settings or by calling `setlocale()`. Recent Windows 10 releases now support a beta option that allows setting the Windows active code page (ACP) to UTF-8 and this does affect the standard library (as well as all other programs running on the system). These additions indicate that it will likely be possible to write native UTF-8 programs for Windows using the Microsoft compiler in the not too distant future. However, there will be existing software written for Windows that will need to be migrated to new C++ standards without incurring the cost of transition to UTF-8 for a long time to come.

IBM has not publicly released a C++11 compliant version of their xLC compiler for z/OS. However, this is due to an intent to migrate to new compiler infrastructure; [IBM XL C/C++ V2.3.1 for z/OS V2.3](#) includes new Clang based compilers [\[ClangOnZ\]](#). [Dignus](#) also provides an LLVM based C++ compiler with support for C++14 and parts of C++17. Though lagging other platforms some, it seems z/OS will continue to remain relevant for new C++ standards.

§ 2.2. Constraint: The ordinary and wide execution encodings are run-time properties

The execution and wide execution encodings are not static properties of programs, and therefore not fully known at compile-time. These encodings are determined at run-time and may be dynamically changed by calls to `setlocale()`. At compile-time, character and string literals are transcoded (in translation phase 5) from the source encoding to an encoding that is expected to be compatible with whatever encoding is selected at run-time. If the compile-time selected encoding turns out not to be compatible with the run-time encoding, then encoding confusion (mojibake) ensues.

The dynamic nature of these encodings is not theoretical. On Windows, the execution encoding is determined at program startup based on the current active code page. On POSIX platforms, the run-time encoding is determined by the LANG, LC_ALL, or LC_CTYPE environment variables. Some existing programs depend on the ability to dynamically change (via POSIX `uselocale()` or Microsoft's `_configthreadlocale()`) the current locale (including the execution encoding) in order for a server process to concurrently serve multiple clients with different locale settings. A recent proposal to WG14 [\(N2226\)](#) [\[WG14-N2226\]](#) proposes allowing the current locale settings to vary by

thread. Attempting to restrict the ability to dynamically change the execution encoding would break existing code.

§ 2.3. Constraint: There is no portable primary execution encoding

On POSIX derived systems, the primary interface to the operating system is via the ordinary execution encoding. This contrasts with Windows where the primary interface is via the wide execution encoding and interfaces defined in terms of `char` are implemented as wrappers around their wide counterparts. Unfortunately, such wrappers are often poor substitutes for use of their wide cousins due to transcoding limitations; it is common that the ordinary execution encoding is unable to represent all of the characters supported by the wide execution encoding.

The designers of the C++17 filesystem library had to wrestle with this issue and addressed it via abstraction; `std::filesystem::path` has an implementation defined `value_type` that reflects the primary operating system encoding. Member functions provide access to paths transcoded to any one of the five standard mandated encodings (ordinary, wide, UTF-8, `char16_t`, and `char32_t`). This design serves as a useful precedent for future design.

§ 2.4. Constraint: `wchar_t` is a portability deadend

The wide execution encoding was introduced to provide relief from the constraints of the (typically 8-bit) `char` based ordinary execution encodings by enabling a single large character set and trivial encoding that avoided the need for multibyte encoding and ISO-2022 style character set switching escape sequences. Unfortunately, the size of `wchar_t`, the character set, and its encoding were all left as implementation defined properties resulting in significant implementation variance. The present situation is that the wide execution encoding is only widely used on Windows where its implementation is actually non-conforming (see <https://github.com/sg16-unicode/sg16/issues/9>).

§ 2.5. Constraint: `char` aliases everything

Pointers to `char` may be used to inspect the underlying representation of objects of any type with the consequence that lvalues of type `char` alias with other types. This restricts the ability of the compiler to optimize code that uses `char`. `std::byte` was introduced in C++17 as an alternative type to use when `char`'s aliasing abilities are desired, but it will be a long time, if ever, before we can deprecate and remove `char`'s aliasing features.

§ 2.6. Constraint: Implementors cannot afford to rewrite ICU

ICU powers Unicode support in most portable C++ programs today due to its long history, impressive feature set, and friendly license. When considering standardizing Unicode related features, we must keep in mind that the Unicode standard is a large and complicated specification, and many C++ implementors simply cannot afford to reimplement what ICU provides. In practice this means that we'll need to ensure that proposals for new Unicode features are implementable using ICU.

§ 2.7. Constraint: File names do not have an associated character encoding

In general, file names do not have an explicit associated encoding. The POSIX [\[POSIX\] definition of "filename"](#) is:

A sequence of bytes consisting of 1 to `{NAME_MAX}` bytes used to name a file. The bytes composing the name shall not contain the `<NUL>` or `<slash>` characters. In the context of a pathname, each filename shall be followed by a `<slash>` or a `<NUL>` character; elsewhere, a filename followed by a `<NUL>` character forms a string (but not necessarily a character string). The filenames **dot** and **dot-dot** have special meaning. A filename is sometimes referred to as a "pathname component". See also Pathname.

and the [definition of "pathname"](#) contains the following note:

If a pathname consists of only bytes corresponding to characters from the portable filename character set (see Portable Filename Character Set), `<slash>` characters, and a single terminating `<NUL>` character, the pathname will be usable as a character string in all supported locales; otherwise, the pathname might only be a string (rather than a character string). Additionally, since the single-byte encoding of the `<slash>` character is required to be the same across all locales and to not occur within a multi-byte character, references to a `<slash>` character within a pathname are well-defined even when the pathname is not a character string. However, this property does not necessarily hold for the remaining characters within the portable filename character set.

It is worth emphasizing that POSIX file names constructed using only characters from the portable filename character set are *usable* in all supported locales, but do not necessarily indicate the same sequence of characters in each locale. Thus, in general, how file names are displayed depends on locale settings.

Some operating systems exhibit strong correlation between file names and a particular encoding. However, it is important to keep in mind that file name restrictions and encoding are determined by both the file system (possibly in conjunction with file system settings and/or mount options) and the

operating system, and that observed conventions do not necessarily indicate enforced requirements. For example, file names on Windows are typically UTF-16 encoded, but NTFS does not enforce well-formed names. The ["Internals" section of the Wikipedia entry for NTFS](#) [\[WikipediaNTFS\]](#) states:

NTFS allows any sequence of 16-bit values for name encoding (file names, stream names, index names, etc.) except 0x0000. This means UTF-16 code units are supported, but the file system does not check whether a sequence is valid UTF-16 (it allows any sequence of short values, not restricted to those in the Unicode standard). ...

This leniency means that valid NTFS file names may contain unpaired surrogate code points and therefore might not be representable as UTF-16, nor therefore be successfully transcodeable to UTF-8. The lack of such restrictions prompted the creation of the [WTF-8 encoding](#) [\[WTF8\]](#).

Windows also natively supports file systems that do not use "wide" file names, for example exFAT, ISO-9660, and NFS. As with POSIX, interpretation of file names on these file systems is locale sensitive.

Unlike POSIX, Windows does not guarantee that the path separator character ('\ U+005C Backslash) exists in all supported locale dependent character sets. As a result, Windows installations configured for Japanese locales will display path separators using the ¥ (U+00A5 Yen) character. Also unlike POSIX, Windows supports "ANSI" encodings that allow 0x5C to appear as a trailing code unit in file names which means that a simple search for the backslash character is insufficient to identify path separators in an "ANSI" encoded file name.

Apple's APFS and HFS+ filesystems require well-formed UTF-8 file names. Additionally, they both support normalization-insensitive file names. APFS stores normalization-preserved file names with an associated hash of the Unicode 9.0 NFD form of the name thereby enabling the file to be opened with a name that doesn't match the original normalization. HFS+ stores file names in Unicode 3.2 NFD form and normalizes when comparing file names. Since the normalization forms used by these filesystems are tied to specific Unicode versions, it is possible for names normalized according to a different Unicode version to fail to match as intended. More information can be found in the ["Frequently Asked Questions - Implementation" section of the Apple File System Guide](#) [\[AppleFSG\]](#).

It is common for C++ programs to produce output that contains file names. It is likewise commonly expected for programs or computer users to be able to extract file names from such output and to be able to open the indicated file directly using the provided name. The requirement to represent file names accurately has profound implications for text processing. It means that, output that is otherwise well-formed text, may be correct, but not well-formed from a text encoding perspective if it contains file names. Likewise, if the output of a program is well-formed text, but is transformed in some way, perhaps transcoded to another encoding or Unicode normalization form, then file names within the text

may be damaged. Similar concerns apply to command lines as well; a program cannot unconditionally expect well-formed text for all of its command line arguments if it accepts file names. These constraints place limits on what an implementation can assume about the input and output of a program.

§ 3. Guidelines: Keep your eyes on the road, your hands upon the wheel

Mistakes happen and will continue to happen. Following a few common guidelines will help to ensure we don't stray too far off course and help to minimize mistakes. The guidelines here are in no way specific to Unicode or text processing, but represent areas where mistakes would be easy to make.

§ 3.1. Guideline: Avoid excessive inventiveness; look for existing practice

C++ has some catching up to do when it comes to Unicode support. This means that there is ample opportunity to investigate and learn from features added to other languages. A great example of following this guideline is found in the [P1097R1](#) [\[P1097R1\]](#) proposal to add named character escapes to character and string literals.

§ 3.2. Guideline: Avoid gratuitous departure from C

C and C++ continue to diverge and that is ok when there is good reason for it (e.g., to enable better type safety and overloading). However, gratuitous departure creates unnecessary interoperability and software composition challenges. Where it makes sense, proposing features that are applicable for C to WG14 will help to keep the common subset of the languages as large as it can reasonably be. [P1041R4](#) [\[P1041R4\]](#) and [P1097R1](#) [\[P1097R1\]](#) are great examples of features that would be appropriate to propose for inclusion in C.

§ 4. Direction: Designing for where we want to be and how to get there

Given the constraints above, how can we best integrate support for Unicode following time honored traditions of C++ design including the zero overhead principle, ensuring a transition path, and enabling software composition? How do we ensure a design that programmers will want to use? The following explores design considerations that SG16 participants have been discussing.

The ordinary and wide execution encodings are not going away; they will remain the bridge that text must cross when interfacing with the operating system and with users. Unless otherwise specified, I/O

performed using `char` and `wchar_t` based interfaces in portable programs must abide by the encodings indicated by locale settings. But internally, it is desirable to work with a limited number of encodings (preferably only one) that are known at compile time, and optimized for accordingly. This suggests a design in which transcoding is performed from dynamically determined external encodings to a statically known internal encoding at program boundaries; when reading files, standard input/output streams, command line options, environment variables, etc... This is standard practice today. When the internal encoding is a Unicode encoding, this external/internal design is sometimes referred to as the [Unicode sandwich](#).

There are two primary candidates for use as internal encodings today: UTF-8 and UTF-16. The former is commonly used on POSIX based platforms while the latter remains the primary system encoding on Windows. There is no encoding that is the best internal encoding for all programs, nor necessarily even for the same program on different platforms. We face a choice here: do we design for a single well known (though possibly implementation defined) internal encoding? Or do we continue the current practice of each program choosing its own internal encoding(s)? Active SG16 participants have not yet reached consensus on these questions.

Use of the type system to ensure that transcoding is properly performed at program boundaries helps to prevent errors that lead to mojibake. Such errors can be subtle and only manifest in relatively rare situations, making them difficult to discover in testing. For example, failure to correctly transcode input from ISO-8859-1 to UTF-8 only results in negative symptoms when the input contains characters outside the ASCII range.

This is where [the `char8\_t` proposal](#) [\[P0482R6\]](#) comes in to play. Having a distinct type for UTF-8 text, like we do for UTF-16 and UTF-32, enables use of any of UTF-8, UTF-16, or UTF-32 as a statically known internal encoding, without the implementation defined signedness and aliasing concerns of `char`, and with protection against accidental interchange with `char` or `wchar_t` based interfaces without proper transcoding having been performed first. Solid support in the type system, combined with statically known encodings, provides the flexibility needed to design safe and generic text handling interfaces, including ones that can support `constexpr` evaluation. Why might `constexpr` evaluation be interesting? Consider [the `std::embed` proposal](#) [\[P1040R1\]](#) and the ability to process a text file loaded at compile time.

Distinct code unit types (`char8_t`, `char16_t`, `char32_t`) enable statically known internal encodings, but not without some cost. Existing code that works with UTF-8 today is written using `char`, `unsigned char`, or `uint8_t`. Likewise, existing code that works with UTF-16 today is written using `char16_t`, `wchar_t`, `unsigned short`, or `uint16_t`. ICU supports customization of its internal code unit type, but `char16_t` is used by default, following ICU's adoption of C++11. Prior to the switch to C++11, the default varied by platform. The switch to `char16_t` created friction with existing code by, for example, requiring that ICU data passed to Windows `wchar_t` interfaces be

copied or `reinterpret_cast`. Similar friction will occur with `char8_t`. ICU dealt with this by providing interfaces that, for at least some cases, encapsulate uses of `reinterpret_cast` and handling of the resulting aliasing issues.

`std::basic_string` isn't a great foundation for working with Unicode text due to its operations all working at the code unit level as opposed to code point or grapheme cluster levels. [The text_view proposal \[P0244R2\]](#) provides a method for layering encoding aware code point support on top of `std::basic_string` or any other string like type that provides a range of code units. SG16 has been discussing the addition of a `std::text` family of types that provide similar capabilities, but that also own the underlying data. Zach Laine has been prototyping such a type in his [Boost.Text library](#).

Introducing new types that potentially compete with `std::string` and `std::string_view` creates a possible problem for software composition. How do components that traffic in `std::string` vs `std::text` interact? Discussions in SG16 have identified several strategies for dealing with this:

1. `std::text` could be convertible to `std::string_view` and, potentially, `const std::string` & if it holds an actual `std::string` object, and
2. `std::text` and `std::string` could allow their buffers to be transferred back and forth (and potentially to other string types).

New text containers and views help to address support for UTF encoding and decoding, but Unicode provides far more than a large character set and methods for encoding it. Unicode algorithms provide support for enumerating grapheme clusters, word breaks, line breaks, performing language sensitive collation, handling bidirectional text, case mapping, and more. Exposing interfaces for these algorithms is necessary to claim complete Unicode support. Exposing these in a generic form that allows their use with the large number of string types used in practice is necessary to enable their adoption. Enabling them to be used with segmented data types (e.g., ropes) is a desirable feature.

§ 5. Directives: Do or do not, there is no try

Per the general design discussion above, the following directives identify activities for SG16 to focus on. Papers exploring and proposing features within their scope are encouraged.

§ 5.1. Directive: Standardize new encoding aware text container and view types

This is the topic that SG16 participants have so far spent the most time discussing, but we do not yet have papers that explore or propose particular designs.

We have general consensus on the following design directions:

- A new `std::text` type that is an owning string type with a statically known character encoding.
- A new `std::text_view` type that is a non-owning string type with a statically known character encoding.
- These types will not have the large interface exposed by `std::string`.
- These types will encourage processing of code points and grapheme clusters while permitting efficient access to code units.

Discussion continues for questions such as:

- Should these types be ranges and, if so, should their `value_type` reflect code points or extended grapheme clusters? Or, should these types provide access to distinct ranges (e.g., via `as_code_points()` and `as_graphemes()` member functions) that require the programmer to explicitly choose a level of abstraction to work at?
- Can these types satisfy the complexity requirements for ranges? Ranges require $O(1)$ for calls to `begin()` and `end()`, but iteration by code point or grapheme cluster may consume an arbitrary number of code units due to ill-formed code unit sequences or arbitrary numbers of combining code points.
- Should these types be comparable via standard operators? If so, should comparison be lexicographical (fast, but surprising if text is not normalized) or be based on canonical equivalence (slower, but consistent results regardless of normalization)? Should a specialization of `std::less` be provided that performs a fast comparison suitable for storing these types in containers?
- Should these types enforce well-formed encoded text? Should validation be performed on each mutation? How should errors be handled?
- Should these types support a single fixed encoding (UTF-8)? Or should multiple encodings be supported as proposed in [the text_view proposal \[P0244R2\]](#)?
- Should these types enforce a normalization form on UTF encoded text?
- Should these types include allocator support?
- Should these types replace use of `std::string` and `std::string_view` in most text/string processing code?

There is much existing practice to consider here. Historically, most string classes have either provided code unit access (like `std::string` or code point access (possibly with means for code unit access as well)). Swift took the bold move of making extended grapheme clusters the basic element of Swift

strings. There are many design options and tradeoffs to consider. Papers exploring the design options are strongly encouraged.

§ 5.2. Directive: Standardize generic interfaces for Unicode algorithms

SG16 participants have not yet spent much time discussing interfaces to Unicode algorithms, though Zach Laine has blazed a trail by implementing support for all of them in his [Boost.Text library](#). Papers exploring requirements would be helpful here. Some questions to explore:

- Is it reasonable for these interfaces to be range based over code points? Or are contiguous iterators and ranges over code units needed to achieve acceptable performance?
- Can these interfaces accommodate segmented data structures such as ropes?
- Many Unicode algorithms require additional context such as the language of the text (Russian, German, etc...). How should this information be supplied? The existing facilities exposed via `std::locale` are more oriented towards abstracting operations than providing this type of information.

§ 5.3. Directive: Standardize useful features from other languages

We've got a start on this with [Named Character Escapes](#) [\[P1097R1\]](#), but there are no doubt many text handling features in other languages that would be desirable in C++. Papers welcome.

§ 5.4. Directive: Improve support for transcoding at program boundaries

C++ currently includes interfaces for transcoding between the ordinary and wide execution encodings and between the UTF-8, UTF-16, and UTF-32 encodings, but not between these two sets of encodings. This poses a challenge for support of the external/internal encoding model.

Portably handling command line arguments (that may include file names that are not well formed for the current locale encoding) and environment variables (likewise) accurately can be challenging. The design employed for `std::filesystem::path` to provide access to native data as well as access to that data transcoded to various encodings could be applied to solve portability issues with command line arguments and environment variables.

An open question is whether transcoding between external and internal encodings should be performed implicitly (convenient, but hidden costs) or explicitly (less convenient, but with apparent costs).

§ 5.5. Directive: Propose resolutions for existing issues and wording improvements opportunistically

While not an SG16 priority, it will sometimes be necessary to resolve existing issues or improve wording to accommodate new features. Issues that pertain to SG16 are currently tracked in our github repo at <https://github.com/sg16-unicode/sg16/issues>.

§ 6. Non-directives: Thanks, but No Thanks

The C++ standard currently lacks the necessary foundations for obtaining or displaying Unicode text through human interface devices. Until that changes, addressing user input and graphical rendering of text will remain out of scope for SG16. Likewise, encodings used for I/O (terminals, stdin, stdout, environment variables, file systems, etc...) are typically determined by the underlying platform that a C++ implementation runs on and cannot be dictated by the C++ standard.

§ 6.1. Non-directive: User input

Keyboard scan codes, key mapping, and methods of character composition entry are all fantastically interesting subjects, but require lower level device access than are currently provided by standard C++. SG16's scope begins at the point where text is presented in memory as an encoded sequence of "characters".

§ 6.2. Non-directive: Fonts, graphical text rendering

What Unicode provides and what fonts and graphical text rendering facilities need are two related but distinct problems. SG16's scope ends at the point where text is handed off to code capable of interacting with output devices like screens, speech readers, and brail terminals.

§ 6.3. Non-directive: Encoding of file names

The permissible set of file names that a program may encounter is determined by the platform (C++ implementation, operating system, and file system) that the program runs on. SG16 is not at liberty to place restrictions on the set of permissible characters or encodings (or lack there of) used to name files.

§ 7. Acknowledgements

SG16 would not exist if not for early and kind encouragement by Beman Dawes.

Thank you to everyone that has attended an SG16 teleconference and has thereby contributed to the discussions shaping our future direction.

§ References

§ Informative References

[AppleFSG]

Apple Inc. [Apple File System Guide](https://developer.apple.com/library/archive/documentation/FileManagement/Conceptual/APFS_Guide/Introduction/Introduction.html). 2018. URL: https://developer.apple.com/library/archive/documentation/FileManagement/Conceptual/APFS_Guide/Introduction/Introduction.html

[ClangOnZ]

IBM. [Benefits of upgrading to XL C/C++ V2.3.1 for z/OS V2.3](https://www-01.ibm.com/support/docview.wss?uid=ibm10879011). 2019. URL: <https://www-01.ibm.com/support/docview.wss?uid=ibm10879011>

[P0244R2]

Tom Honermann. [text_view: A C++ Concepts and Range based Character Encoding and Code Point Enumeration Library](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0244r2.html). 2017. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0244r2.html>

[P0482R6]

Tom Honermann. [char8_t: A type for UTF-8 characters and strings \(Revision 6\)](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0482r6.html). 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0482r6.html>

[P1025R1]

Steve Downey, et al.. [Update The Reference To The Unicode Standard](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1025r1.html). 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1025r1.html>

[P1040R1]

JeanHeyd Meneide. [std::embed](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1040r1.html). 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1040r1.html>

[P1041R4]

Martinho Fernandes. [Make char16_t/char32_t String Literals be UTF-16/32](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1041r4.html). 2019. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1041r4.html>

[P1072R3]

Chris Kennelly; Mark Zeren. [basic_string::resize_default_init](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1072r3.html). 2019. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1072r3.html>

[P1097R1]

Martinho Fernandes. [Named Character Escapes](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1097r1.html). 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1097r1.html>

[P1238R0]

Tom Honermann, et al.. [SG16: Unicode Direction](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1238r0.html). 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1238r0.html>

[POSIX]

IEEE and The Open Group. [The Open Group Base Specifications Issue 7, 2018 edition, IEEE Std 1003.1-2017](http://pubs.opengroup.org/onlinepubs/9699919799/1003.1-2017). 2018. URL: [http://pubs.opengroup.org/onlinepubs/9699919799/](http://pubs.opengroup.org/onlinepubs/9699919799/1003.1-2017)

[W3Techs]

W3Techs. [Usage of UTF-8 for websites](https://w3techs.com/technologies/details/en-utf8/all/all). 2017. URL: <https://w3techs.com/technologies/details/en-utf8/all/all>

[WG14-N2226]

Florian Weimer. [Optional thread storage duration for the program's locale](http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2226.htm). 2018. URL: <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2226.htm>

[WikipediaNTFS]

[NTFS](https://en.wikipedia.org/wiki/NTFS). 2019. URL: <https://en.wikipedia.org/wiki/NTFS>

[WTF8]

Simon Sapin. [The WTF-8 encoding](https://simonsapin.github.io/wtf-8/). 2018. URL: <https://simonsapin.github.io/wtf-8/>